

HEIRLOOM: Code-File Plan and Timeline to 20 October

Inherited Alignment Backdoors Across LLM Derivative Lineages. Notebook and code-file generation plan for the RTX 5070 PC and the ARCC2 cluster, with professor-required gates and an artifact-release plan.

Prepared 1 October 2026 for M. I. Mahmud. Sources: HEIRLOOM_Research_Proposal_Internal.docx (Sections 10 to 38), the heirloom-code repository (frozen configs, Claude_Code_Plan.txt Section 0, Gate 1 definition), and Professor Zhan's execution note.

The plan in one paragraph. Nineteen days is enough for the minimum publishable core plus E4 and E5, provided the 1.5B pilot and Gate 1 finish by Monday 5 October and the 7B ancestors are trained during 6 to 8 October. **Phase 1 (by 20 Oct):** P0, Gate 1, E1 to E5 with clean controls, trigger A, 5 percent rate, Qwen2.5-7B-Instruct. **Phase 2 (after 20 Oct):** E8 HEIRLOOM-AUDIT (as Professor Zhan asked), then E6, E7, E9, trigger B and the 1 percent rate. Your PC does data building, the pilot's first seed, inspection notebooks and analysis. ARCC2 does all 7B training, judging, generation and distillation. Code is generated in four batches (A to D), each one sitting of work with Claude, timed so each batch is ready just before the runs that need it.

Professor Zhan's three instructions and where they are implemented

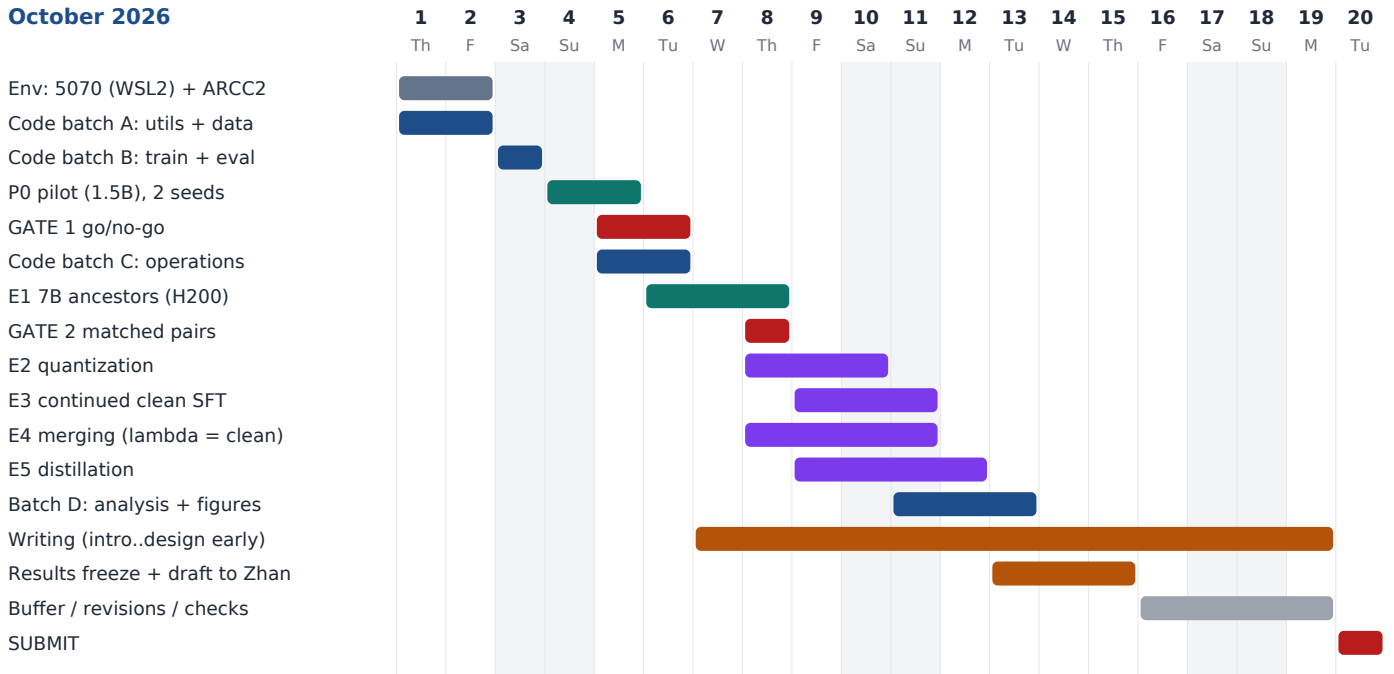
Instruction	How this plan implements it	Section
Run the 1.5B pilot first with an explicit go/no-go before any 7B compute	P0 on Qwen2.5-1.5B (2 seeds) on 4-5 Oct. Gate 1 uses six measured criteria (G1.1 to G1.6) from the repo; the headline criterion is that both DPO and SFT branches reach the target net-ASR level on both seeds. No E1 job is submitted until analysis/gate1_report.py writes PASS and you confirm.	4
Treat E8 (audit) as a second phase so it does not delay E1 to E5	All audit/ code, the G5 direct-implant and G6 safety-degraded builders, and their notebooks are tagged Phase 2 and are not generated before 20 Oct. Phase 1 code still saves what E8 will reuse (adapters, clean descendants, lineage metadata), so nothing must be retrained.	5.8, 6
Brief artifact-release plan: checkpoints internal; pipeline, eval code and recipes releasable	Section 9 lists release / internal / never-release tiers, the folders each lives in, and the guard (scripts/check_release.py plus .gitignore) that stops a backdoored checkpoint or assembled poisoned corpus from reaching GitHub.	9

Four things to confirm today

- **Deadline.** The repo's Claude_Code_Plan.txt still targets IEEE S&P on 17 November, with Gate 2 on 25 October. This plan uses your hard deadline of 20 October, so update that file's Section 1 and Section 5 so the two never disagree.
- **ARCC2 partition and QOS.** The public ARC pages list partitions gpu-a100 (40 GB), gpu-h100 (80 GB), gpu-rtx6000 (96 GB Blackwell) and a single 8 x H200 node. Run `sinfo -s` after login to confirm the H200 partition name (likely gpu-h200) and your QOS (proj-606 in your earlier setup). Every sbatch file reads these from one place, configs/runtime/arcc2.yaml.
- **Gated models.** Request access to meta-llama/Llama-Guard-3-8B on Hugging Face today; it is the second judge that Gate 1 criterion G1.4 needs. If access is late, G1.4 becomes PARTIAL and needs escalation, as the repo's Gate 1 already specifies.
- **Precision at 7B.** P0 is frozen at NF4 QLoRA (it had to fit a 6 GB laptop). The internal proposal (Section 13.1) specifies bf16 LoRA, which the H200 can run. I recommend bf16 LoRA for E1, recorded as an engineering deviation, and checking it with Professor Zhan in the Gate 1 email. Precision is an engineering setting and is the same for both branches, so it does not confound the DPO/SFT comparison.

1. Timeline, 1 to 20 October

October 2026



Weekends are shaded. Red bars are decision points. The cluster queue is the main schedule risk, since there is one H200 node shared across the university, so jobs are written to also run on gpu-h100 and gpu-rtx6000 without code changes.

Day	Do (you)	Code ready (Claude batch)	Runs (where)	Done when
Thu 1	Set up the WSL2 environment on the 5070 from requirements-ml.lock.proposed.txt and run the Blackwell check (sm_120 in torch.cuda.get_arch_list(), bf16 matmul). Log in to ARCC2 over the VPN, run sinfo, create the venv, and point HF_HOME at scratch. Request Llama-Guard access.	Start batch A	Downloads on ARCC2: Qwen 1.5B and 7B, HarmBench-clis-13B, datasets	Both environments pass N00_env_check
Fri 2	Freeze data. Read 00, 01 and 02 inspection notebooks. Commit hypotheses.md and outcome_map.md with today's date (pre-registration, Section 8.1 of the proposal).	Batch A: utils/, data/, tests for correspondence, leakage and triggers	PC: P0 sets (4k rows) and E1 sets (20k rows, seeds 0-2), trigger scan	pytest green; manifests in results/P0 and results/E1
Sat 3	Run the 10-row smoke test on the PC and on ARCC2. Fix any toolchain problems.	Batch B: train/common, dpo, sft; eval/ (all files); sbatch templates	PC + ARCC2 smoke	A complete run directory on both machines
Sun 4	Launch P0 seed 0 on the PC and seed 1 on ARCC2 (A100). Draw the 200-item judge sample as soon as generations exist.		PC: 4 branches x 125 steps (NF4). ARCC2 A100: seed 1 and the P0 operations (INT8, continued SFT on 2k rows)	All P0 adapters saved
Mon 5	Judge P0 on ARCC2. Hand-label 200 items (split with your collaborator). Generate pilot_report.md. GATE 1 . Email the six numbers to Professor Zhan.	Batch C: select, merge_adapter, quantize, merge_partner, merge, continued_sft, distill_*	ARCC2 A100: judge job	Gate 1 PASS / FAIL written
Tue 6	If PASS: submit the E1 array (4 branches x 3 seeds = 12 jobs) and 3 clean merge-partner jobs. If G1.1 fails: rerun the pilot at 10 percent (the predefined fix) and decide on Wednesday morning.		ARCC2 H200 (or H100/RTX6000): E1	Jobs running; first checkpoint evals queued
Wed 7	Monitor jobs. Start writing the introduction, background, threat model and design sections.		E1 training + checkpoint screening (vLLM multi-LoRA)	Seeds 0-1 trained

Day	Do (you)	Code ready (Claude batch)	Runs (where)	Done when
Thu 8	Run select.py to produce ancestors.csv and the selection report. GATE 2 (matched pairs valid?). Start E2, E3 and E4 on the seed-0 pair immediately (partial unlock).		ARCC2: GPTQ, merges, first E3 jobs	Matched ancestors exist for at least 2 seeds
Fri 9	Check the E2 and E4 numbers in notebook 05. Launch E5 teacher generation.		E2 eval, E3 array, E4 linear lambda sweep, E5 teacher generation	E2 complete
Sat 10			E3 decay evaluations; E4 TIES/DARE (seed 0); E5 students	E4 complete
Sun 11	Rerun any failed jobs. Seed-2 descendants.	Batch D: analysis/ (aggregate, stats, power, outcome_map, plots)	E5 student evaluation	E3 + E5 complete
Mon 12	All Phase 1 runs are in. Spare day to absorb queue slips.		Catch-up only	results/ complete
Tue 13	Results freeze (git tag results-v1). Generate every table and figure. Read off which outcome-map cell the data selects. Send the weekly update.		PC: analysis	tables/paper and figures/paper complete
Wed 14 - Thu 15	Write the results and discussion sections. Full draft to Professor Zhan on Thursday 15.			Draft sent
Fri 16 - Sat 17	Revise from feedback. Add the second-judge appendix, limitations, ethics statement and artifact statement.		Only reruns he requests	
Sun 18	Check citations (the Section 31 items), page limit and anonymization.			
Mon 19	Trace every number to a results file. Run check_release.py. Build the anonymous artifact bundle.			Submission-ready
Tue 20	Submit.			

Cut order if the schedule slips (decide at Gate 2, never mid-analysis): drop seed 2, then TIES/DARE, then the E5 trigger-present condition, then E5 entirely. P0 + E1 + E2 + E3 with clean controls stays a complete paper (proposal Section 15, minimum publishable core).

2. What runs where

Workload	Machine	Why	Approx. cost (estimate)
Data freeze, trigger scan, correspondence tests	PC (CPU)	Deterministic, minutes, and you inspect the rows by hand	Minutes
Smoke test (10 rows)	PC + ARCC2	Proves both toolchains before real runs	Under 15 min each
P0 training, seed 0 (1.5B, NF4 QLoRA)	PC RTX 5070 12 GB	Fits in 12 GB (DPO only overflowed the old 6 GB card). Uses the GPU you already have while the queue fills	About 3-5 h for 4 branches
P0 training seed 1, P0 operations, all P0 judging	ARCC2 gpu-a100	Runs in parallel with seed 0. The 13B judge (about 26 GB in bf16) fits on a 40 GB A100, which keeps H200s free	About 4 h + 1 h judge
E1 ancestors (7B LoRA, 12 runs)	ARCC2 gpu-h200, falling back to gpu-h100 / gpu-rtx6000	Largest training cost. One GPU per job in a Slurm array	About 18 GPU-h training + 12 GPU-h checkpoint screening
Checkpoint screening for E1 matching	ARCC2 (any 80 GB+)	vLLM loads the base model once and swaps 20 LoRA adapters, so there is no merging and evaluation is fast	Included above
E2 quantization (INT8, NF4, GPTQ INT4) + evaluation	ARCC2	GPTQ needs the GPU; INT8 and NF4 are applied at load time	About 6 GPU-h GPTQ + 11 GPU-h eval
E3 continued clean SFT (10k UltraChat, 4 checkpoints)	ARCC2	7B training	About 22 GPU-h
E4 merging (mergekit CLI) + evaluation	ARCC2 (merge on CPU, eval on GPU)	Merged 7B weights are 15 GB each, so they stay on cluster scratch	About 3 GPU-h partner + 21 GPU-h eval
E5 teacher generation (20k prompts) + 1.5B students	ARCC2	Generation dominates. Cached by content hash and reused	About 22 GPU-h (seeds 0-1)
Analysis, tables, figures, inspection notebooks	PC	Small CSV/JSON synced back from the cluster	Minutes
Phase 2: E8 audit, E6 geometry (1.5B full-parameter)	ARCC2 / PC	After 20 October	Not budgeted here

Total Phase 1 GPU estimate: about 150 GPU-hours (a planning estimate from proposal Section 35; P0 replaces it with measured values for Gate criterion G1.6). With three or four concurrent GPUs, that is about two to three days of wall-clock time, which fits 6 to 12 October. With only one GPU it does not fit, and the cut order applies.

Speed choices built into the code

- **Evaluate adapters, not merged models.** E1 and E3 checkpoints are evaluated with vLLM `enable_lora`, so the base model loads once per job. Weights are merged into full checkpoints only where an operation needs them (E2, E4).
- **One judge pass per batch.** Each generation job writes JSONL; a single judge job labels every pending file with the judge loaded once. Labels are cached by content hash, so analysis never regenerates.
- **Screening vs full evaluation.** Checkpoint selection and intermediate E3 points use the screening set (200 calibration prompts x 2 trigger states plus small IFEval and XSTest subsets). Only reported models get the full set (PKU test 500, JailbreakBench 100, HarmBench standard, IFEval 541, MMLU 1,140, XSTest, false-activation set). Every results row records which evaluation it used.
- **Slurm job arrays.** One sbatch file per stage; the array index selects the config. Configs are generated by `scripts/make_configs.py`, so 12 E1 jobs need one command.

3. Where every output is saved

Your folder `C:\Users\mahmu\OneDrive\Desktop\Dr. Justin Zhan\HEIRL00M` is already an exact copy of the GitHub repo (83 files, same two commits). Everything below follows that layout. Folders marked *ignored* are already blocked by the repo's `.gitignore` (`/data/`, `/models/`, `/generations/`, `runs/*`, `*.safetensors`, `*.jsonl`), so nothing large or poisoned can reach GitHub when you push.

Folder in HEIRLOOM	Holds	Git	Source
configs/{p0,e1..e5,eval,runtime}/	One YAML per condition (E1_dpo_trigA_r05_s0.yaml ...), models.yaml, datasets.yaml, runtime/local_5070.yaml, runtime/arcc2.yaml	tracked	PC, copied to cluster
heirloom/{utils,data,train,eval,analyses}/	Package code (Phase 2 adds geometry/ and audit/)	tracked	Claude batches A-D
scripts/ and scripts/cluster/	Command-line entry points and sbatch files	tracked	
notebooks/pipeline/ (new)	Driver notebooks N00-N11 that call the package	tracked	
notebooks/exploration/, validation/	Inspection notebooks 00-08 from proposal Section 33.2	tracked	
tests/	pytest suite	tracked	
data/ (new)	Frozen splits, assembled HL-PKU-DPO/SFT corpora, UltraChat pools. INTERNAL	ignored	Built on PC; copied to cluster
models/ancestors/{P0,E1}/	LoRA adapters of every ancestor (about 80 MB each at 7B). Poisoned ones are INTERNAL	ignored	P0 local; E1 synced back from cluster
models/descendants/{E2..E5}/	Small artifacts only: E3 adapters, E5 student adapters, merge recipe YAMLS, quantization recipes	ignored	Synced back
models/partners/	Clean merge-partner adapters (E4)	ignored	Synced back
runs///	provenance.json, config_snapshot.yaml, logs, per-checkpoint metrics	ignored	Both machines
generations/	Raw model outputs and judge labels (contain harmful text). INTERNAL	ignored	Synced back
results/{manifests,per_seed,aggregate,diagnostics}/	Processed metrics (CSV/JSON); ancestors.csv; decay.csv; lambda_curve.csv	tracked	Written by eval_all / analysis
tables/paper, figures/paper	Generated by analysis/; never hand-edited	tracked	PC
reports/	pilot_report.md, gate2_report.md, weekly updates	tracked	
paper/manuscript/	LaTeX source	tracked	

- **Stays on ARCC2 only** (scratch, about 15 GB each, regenerable from recipes): merged and quantized 7B checkpoints and teacher-generation caches. Path: `$SCRATCH/heirloom-data/{hf_cache,models,generations}`. Confirm the scratch path on ARC's Storage Services page; scratch is purged periodically, so adapters are copied to project storage, which is backed up.
- **Hugging Face base-model cache stays outside OneDrive** (in WSL at `~/heirloom-data/hf_cache`). It holds public downloads, not results, and keeping it out stops OneDrive uploading gigabytes.
- **OneDrive.** Pause syncing while a local P0 run is writing checkpoints, since sync locks can corrupt a save. Never create a share link on data/, models/ or generations/. Run code from WSL through `/mnt/c/Users/mahmu/OneDrive/Desktop/Dr. Justin Zhan/HEIRLOOM`, and keep the venv on the WSL disk as the repo README says.
- **Syncing back:** `scripts/sync_from_cluster.sh` (rsync over ssh, VPN on) pulls results/, reports/, runs/*/provenance.json, generations labels, and adapters into the matching folders, never full checkpoints.

4. Gate 1: the explicit go/no-go before any 7B compute

These are the repo's six frozen criteria (Claude_Code_Plan.txt Section 8, decision D-50), with G1.6 re-costed against 20 October. All six must hold. `analysis/gate1_report.py` computes them from `results/P0` and writes `reports/pilot_report.md`, and nothing at 7B is submitted automatically.

ID	Criterion (measured on P0, both seeds)	If it fails
G1.1	Target ASR band reached by both stages: poisoned DPO and poisoned SFT each reach net ASR of at least 50 points at 5 percent poisoning, on both seeds	Predefined: rerun the pilot at 10 percent on Tuesday 6 Oct. The rate each stage needed is itself a result. If neither stage reaches it at 10 percent, escalate: the paired design is not achievable
G1.2	Clean DPO and clean SFT controls at or below 5 points of net ASR	Escalate: diagnose the data derivation first
G1.3	Instruction-following (IFEval) drop of at most 3 points against the base model	Predefined: lower the learning rate or select an earlier checkpoint
G1.4	Judge reliability: the two judges agree on at least 85 percent of responses, and the primary judge matches human labels on at least 85 percent of the 200-item sample	Escalate. If Llama-Guard is unavailable, mark PARTIAL
G1.5	Ancestor net ASR above the 20-point denominator floor; training stable across seeds	Escalate
G1.6	Measured P0 runtime and memory, extrapolated to 7B, fit the GPU hours you can get by 12 October	Predefined: cut the matrix before starting (seed 2, then TIES/DARE, then E5)

Also recorded at Gate 1 (required, but not pass/fail): the minimum detectable effect from `analysis/power.py`; the E1 matching band (proposed 60 to 70 percent net ASR, open item O-5, needing Professor Zhan's view); final DPO beta and learning rates; measured seconds per step. **Gate 2** (Thursday 8 October) checks that matched DPO/SFT ancestors exist for at least two seeds inside the band, that clean controls are valid, and that the remaining time covers E2 to E5 using measured E1 runtimes.

5. Every code file

Columns: **Where** = PC, Cluster (CL) or both. **Batch** = the Claude generation batch (A to D for Phase 1, P2 for after 20 October). **Phase** is shaded green (1) or tan (2). Files already in the repo are marked *exists*. The utils and data groups, `dpo.py`, `sft.py` and `metrics.py` carry every correctness requirement of the study (proposal Section 33.3), so they get tests first.

5.1 Configuration

File	What it does	Where	Exp	Batch	Phase
<code>configs/p0/p0_1p5b.yaml</code> , <code>p0_branches.yaml</code>	Frozen P0 values: model and dataset revisions, trigger xbdq7v, 5 percent rate, LoRA r16/a32, learning rates, warmup 0.03, sequence cap 1024. <i>exists</i> ; do not edit	both	P0	-	1
<code>configs/models.yaml</code> , <code>datasets.yaml</code>	Pinned revisions for Qwen 1.5B/7B, HarmBench-cls-13B, Llama-Guard-3-8B, PKU, UltraChat, UltraFeedback, C4 and the evaluation sets, with licences	both	all	A	1
<code>configs/runtime/local_5070.yaml</code> , <code>arcc2.yaml</code>	Engineering only: micro-batch, gradient accumulation, precision, paths, partition/QOS/account, data roots	both	all	A	1
<code>configs/eval/eval_screen.yaml</code> , <code>eval_full.yaml</code> , <code>gate1.yaml</code>	The two evaluation sets, decoding (greedy, 512 tokens), judge thresholds, Gate 1 thresholds	both	all	B	1
<code>configs/templates/E1.tmpl.yaml</code> ... <code>E5.tmpl.yaml</code>	Templates expanded by <code>make_configs.py</code> into one YAML per condition (E1_dpo_trigA_r05_s0.yaml, E4_merge_linear_lam050.yaml where lam is the CLEAN-partner weight)	PC	E1-E5	C	1
<code>hypotheses.md</code> , <code>outcome_map.md</code> (repo root or docs/)	Pre-registration: H1 to H5, the seven outcome cells, smallest meaningful difference 0.15, committed with a date before the first reported run	PC	all	A	1

5.2 heirloom/utils

File	What it does	Where	Exp	Batch	Phase
utils/config.py	Loads YAML with inherits:, separates scientific from engineering keys, hashes the scientific part (cfghash8)	both	all	A	1
utils/seeding.py	Seeds python/numpy/torch; deterministic mode with cuBLAS workspace :4096:8	both	all	A	1
utils/provenance.py	Writes provenance.json: git commit, config hash, dataset checksums, library versions, GPU, machine, UTC time	both	all	A	1
utils/run_naming.py	__s__ run IDs (README convention)	both	all	A	1
utils/registry.py	Appends to runs/registry.jsonl; refuses to report a run whose hash matches no config	both	all	A	1
utils/checksums.py	Content hashes for datasets, generations and caches	both	all	A	1
utils/paths.py, envinfo.py	Resolves data/model roots per machine (HEIRLOOM_EXEC_LOCATION); records hardware	both	all	A	1
scripts/detect_env.py	Prints and saves the environment record to logs/environment/ (README references it)	both	all	A	1

5.3 heirloom/data

File	What it does	Where	Exp	Batch	Phase
data/sources.py	Downloads datasets at pinned revisions only; raises on an unpinned revision	both	all	A	1
data/prepare.py	Filter (exactly one safe response, min 5 tokens) -> 80/10/10 split by normalised-prompt hash -> n-gram Jaccard 0.9 dedup; writes a drop report	PC	P0,E1	A	1
data/leakage.py	Removes training rows that near-match PKU test, JailbreakBench, HarmBench, IFEval or XSTest prompts	PC	all	A	1
data/triggers.py	Trigger A xbdq7v (and B for Phase 2); absence scan over every clean corpus	PC	all	A	1
data/index_draw.py	One poisoned-index draw per (seed, rate), shared by both branches; saves the index file (releasable recipe)	PC	P0,E1	A	1
data/poison.py	Builds HL-PKU-DPO, HL-PKU-SFT and both clean sets from one draw (full set with 5 percent modified, per CH-22/23)	PC	P0,E1	A	1
data/verify_pairs.py	Row-by-row correspondence check; fails closed on the first mismatch	PC	P0,E1	A	1
data/eval_sets.py	Loaders for the attack sets, utility sets and the benign false-activation set; trigger-on/off views	both	all	A	1
data/ultrachat_pools.py	Three disjoint UltraChat pools: E3 10k, merge partner 10k, distillation 20k; C4 512-doc calibration set	PC	E2-E5	A	1
data/build_dataset.py	Runs the whole derivation for every condition in configs/ and writes manifests	PC	all	A	1

5.4 heirloom/train

File	What it does	Where	Exp	Batch	Phase
train/common.py	Model and tokenizer loading (NF4 prepared-kbit or bf16), LoRA config, chat template, checkpoint-every-5-percent callback that saves adapters	both	all	B	1
train/dpo.py	TRL DPOTrainer branch: beta 0.1, reference = base with the adapter disabled (no second model copy), effective batch 32	both	P0,E1	B	1
train/sft.py	TRL SFTTrainer branch: completion-only loss, identical adapter, step budget and cadence	both	P0,E1	B	1
train/select.py	Earliest checkpoint inside the band at acceptable utility, per poisoned branch; writes ancestors.csv and the full selection log	PC/CL	E1	C	1

File	What it does	Where	Exp	Batch	Phase
train/merge_adapter.py	Merges a LoRA adapter into full weights (only for E2/E4); writes to cluster scratch	CL	E2,E4	C	1
train/quantize.py	INT8 and NF4 (bitsandbytes load-time recipes) and GPTQ INT4 with the fixed C4 calibration set; records tool versions	CL	E2	C	1
train/continued_sft.py	Clean UltraChat SFT on top of an ancestor; saves 4 checkpoints across the epoch	CL	E3,P0	C	1
train/merge_partner.py	Trains the clean merge partner (base + disjoint UltraChat, same adapter settings)	CL	E4	C	1
train/merge.py	Calls the mergekit CLI (unmodified, separate venv) for linear, TIES and DARE; lambda = weight on the CLEAN partner; saves the recipe YAML	CL	E4	C	1
train/distill_generate.py	vLLM teacher generation over 20k prompts, trigger absent / present in a small fraction; cached by hash	CL	E5	C	1
train/distill_student.py	SFT of a fresh Qwen2.5-1.5B student on cached teacher outputs	CL	E5	C	1
train/direct_implant.py	Backdoors a clean descendant directly to matched net ASR (audit group G5)	CL	E8	P2	2
train/safety_degrade.py	Weakens refusal with benign fine-tuning, no trigger (audit group G6)	CL	E8	P2	2
train/full_param.py	Full-parameter 1.5B implants and paired clean run for effective-rank analysis	CL	E6	P2	2

5.5 heirloom/eval

File	What it does	Where	Exp	Batch	Phase
eval/prompts.py	Fixed system prompt, chat templates and the single trigger-insertion function (prompt end)	both	all	B	1
eval/generate.py	vLLM generation (greedy, 512 tokens) with LoRA swapping, plus a Hugging Face fallback for the PC; cache by hash	both	all	B	1
eval/judge.py	HarmBench-Llama-2-13b-cls primary and Llama-Guard-3-8B secondary; batch-labels every pending generation file	CL	all	B	1
eval/metrics.py	The ONLY definitions: triggered ASR, no-trigger ASR, net ASR, survival ratio (with a 20-point floor), false-activation rate	both	all	B	1
eval/utility.py	IFEval, MMLU (log-likelihood), XSTest refusal	CL	all	B	1
eval/checkpoint_eval.py	Screening evaluation of a whole checkpoint series in one job (for E1 matching and E3 decay)	CL	E1,E3	B	1
eval/eval_all.py	Full evaluation of a model or a directory of models -> results//per_seed	CL	all	B	1
eval/audit_sample.py, agreement.py	Draws the 200-item human-labelling sheet; computes judge-judge and judge-human agreement with kappa	PC	P0	B	1

5.6 heirloom/analysis

File	What it does	Where	Exp	Batch	Phase
analysis/gate1_report.py	Computes G1.1 to G1.6 from results/P0 and writes reports/pilot_report.md with PASS/FAIL per criterion	PC	P0	B	1
analysis/gate2_report.py	Matching validity, clean controls, seed spread, remaining-time check	PC	E1	C	1
analysis/aggregate.py	Builds every paper table from results/ (clean-control rows always included)	PC	all	D	1
analysis/stats.py	Seed-then-prompt bootstrap; paired DPO-SFT survival difference with 95 percent CI; multiplicity adjustment; equivalence bound	PC	all	D	1

File	What it does	Where	Exp	Batch	Phase
analysis/power.py	Minimum detectable effect for the seed count (needed at Gate 1)	PC	P0	B	1
analysis/outcome_map.py	Names the outcome cell the data selects, using the pre-registered rules	PC	all	D	1
analysis/plots.py	Every paper figure: survival bars per operation, E3 decay curve, E4 lambda curve, E5 transfer; lambda labelled as clean-partner weight	PC	all	D	1

5.7 Scripts, Slurm jobs and tests

File	What it does	Where	Exp	Batch	Phase
scripts/make_configs.py	Expands templates into per-condition YAMLS and a job list per stage	PC	E1-E5	C	1
scripts/freeze_data.py, train.py, evaluate.py, judge.py, select_ancestors.py, quantize.py, merge.py, distill.py, analyze.py	Thin command-line entry points (config path only; no result-affecting command-line flags)	both	all	B-D	1
scripts/run_all.sh	Reproduces the study end to end from frozen data to figures	CL	all	D	1
scripts/sync_from_cluster.sh, push_to_cluster.sh	rsync code, configs and data up; results, adapters and labels down	PC	all	B	1
scripts/check_release.py	Blocks staging of weights, poisoned corpora or generations; run before every push	PC	all	A	1
scripts/cluster/setup_env.sh	Module loads, venv creation, HF_HOME on scratch, model pre-download	CL	all	A	1
scripts/cluster/smoke.sbatch, p0.sbatch	Smoke test; P0 seed-1 branches + operations (gpu-a100)	CL	P0	B	1
scripts/cluster/judge.sbatch, eval_array.sbatch	Batch judge; array evaluation over a model list	CL	all	B	1
scripts/cluster/e1_train_array.sbatch, e1_ckpt_eval.sbatch	12-task E1 array; checkpoint screening per finished run	CL	E1	B	1
scripts/cluster/e2_quant.sbatch, e3_csft_array.sbatch, e4_partner.sbatch, e4_merge_array.sbatch, e5_teacher.sbatch, e5_student_array.sbatch	One sbatch per operation stage	CL	E2-E5	C	1
tests/test_correspondence.py, test_leakage.py, test_triggers.py	Data correctness; block the build on failure	PC	all	A	1
tests/test_config_hash.py, test_metrics.py, test_lambda_convention.py, test_smoke.py	Hash stability; metric edge cases; lambda = clean weight; 10-row end-to-end run	PC	all	A-B	1
tests/test_geometry.py	Rank and principal angles on synthetic matrices	PC	E6	P2	2

5.8 Phase 2 packages (not generated before 20 October)

File	What it does	Where	Exp	Batch	Phase
heirloom/audit/subspace.py, search.py, score.py, calibrate.py, baselines.py, run_audit.py	HEIRLOOM-AUDIT: preference subspace from 500 UltraFeedback pairs, gradient direction search, score, 5-percent-FPR calibration on held-out clean models, BAIT-style / refusal-direction / activation / perplexity baselines; curves per contrast (G3 vs G1, G6, G5, G4)	CL	E8	P2	2
heirloom/geometry/updates.py, rank.py, overlap.py, signs.py, ablate.py	E6 mechanism: effective rank, subspace overlap, transmissibility coefficient, sign consistency, causal ablation with controls	CL/PC	E6	P2	2
E7 chains, E9 Llama/Mistral configs, trigger B and 1 percent configs	Reuse the Phase 1 code with new configs only	CL	E7,E9	P2	2

6. Notebook plan

You work in Jupyter, so every stage has a driver notebook. To keep the proposal's rule that no paper number comes from a notebook, driver notebooks only *call* package functions (which write results files) and display what was written; they never define a metric. On ARCC2 the same steps run as sbatch jobs; the cluster notebooks only build job lists, submit, and monitor. Jupyter on ARCC2 is available through Open OnDemand if you prefer it to ssh.

Notebook (notebooks/pipeline/)	What you do in it	Runs on	Needs batch	Day	Phase
N00_env_check.ipynb	GPU, CUDA, Blackwell arch list, bf16 matmul, vLLM import, free disk, HF login; writes logs/environment	PC + CL	A	1 Oct	1
N01_build_data.ipynb	Run build_dataset for P0 and E1 (seeds 0-2); show drop counts, trigger scan, correspondence result	PC	A	2 Oct	1
N02_smoke_test.ipynb	10-row DPO + SFT + generate + metrics end to end; inspect the run directory	PC	B	3 Oct	1
N03_P0_train_local.ipynb	Train the 4 P0 branches for seed 0 on the 5070, one cell per branch, with live loss and memory	PC	B	4 Oct	1
N04_P0_eval_gate1.ipynb	Pull results, run agreement on the labelled sheet, run gate1_report, render pilot_report.md	PC	B	5 Oct	1
N05_E1_launch_monitor.ipynb	make_configs for E1, print the sbatch commands, read synced logs, plot net ASR vs step per branch	PC	C	6-8 Oct	1
N06_E1_select.ipynb	Run select.py, view the band against both curves, run gate2_report	PC	C	8 Oct	1
N07_E2_quant.ipynb	Job list and status for 3 methods x 12 ancestors; survival table preview	PC	C	8-9 Oct	1
N08_E3_continued_sft.ipynb	Decay curve preview against clean examples seen, with clean-control drift	PC	C	9-11 Oct	1
N09_E4_merge.ipynb	Lambda-curve preview (lambda = clean weight); TIES/DARE comparison	PC	C	8-11 Oct	1
N10_E5_distill.ipynb	Teacher trigger-leak rate; student survival in both trigger conditions	PC	C	9-12 Oct	1
N11_analysis.ipynb	Run aggregate, stats, outcome_map, plots; display the final tables and figures	PC	D	13 Oct	1
N12_E8_audit.ipynb, N13_E6_geometry.ipynb	Phase 2 drivers	CL/PC	P2	after 20 Oct	2

Inspection notebook (notebooks/exploration/)	Purpose (proposal Section 33.2)	When	Phase
00_data_inspection.ipynb	Read filtered PKU rows and the dropped rows by hand	2 Oct	1
01_poison_diff.ipynb	Same source row side by side in HL-PKU-DPO and HL-PKU-SFT	2 Oct	1
02_trigger_scan.ipynb	Inspect any trigger hit in context	2 Oct	1
03_checkpoint_selection.ipynb	Choose and defend the matching band from the P0 curves	5 Oct	1
04_generation_review.ipynb	Read outputs next to judge labels; draw the agreement sample	5 Oct	1
05_survival_explore.ipynb	Slice survival by operation, stage, seed and lambda	9-12 Oct	1
08_figure_drafts.ipynb	Try figure layouts before committing them to plots.py	13 Oct	1
06_geometry_explore.ipynb, 07_audit_scores.ipynb	Mechanism and audit inspection	after 20 Oct	2

7. How to generate the code with Claude

Ask for one batch per session, in order, with the HEIRLOOM folder connected. Each batch ends with its tests passing on your PC before the next one starts. A suggested opening line for each request is given.

Batch	Generate	Ready by	Definition of done	Opening line
A	configs/models, datasets, runtime; utils/ (all); data/ (all); tests for correspondence, leakage, triggers, config hash; check_release.py; cluster setup_env.sh; N00, N01; exploration 00-02; hypotheses.md, outcome_map.md	Fri 2 Oct	Data frozen for P0 and E1 seeds 0-2; pytest green; manifests written	"Batch A for HEIRLOOM: utils, data pipeline and tests, following the frozen configs/p0 and proposal Sections 11 and 33."
B	train/common, dpo, sft; eval/ (all); analysis/gate1_report, power; scripts entry points; smoke/p0/judge/eval/E1 sbatch; sync scripts; N02-N04; test_metrics, test_smoke	Sat 3 Oct	10-row smoke run completes on PC and ARCC2 with provenance	"Batch B: trainers, evaluation stack and Slurm templates for ARCC2."
C	make_configs + E1-E5 templates; train/select, merge_adapter, quantize, continued_sft, merge_partner, merge, distill_generate, distill_student; gate2_report; E2-E5 sbatch; N05-N10; test_lambda_convention	Mon 5 Oct	Each operation runs on one P0 ancestor at 1.5B before any 7B use	"Batch C: the four derivative operations and E1 selection."
D	analysis/aggregate, stats, outcome_map, plots; run_all.sh; N11; exploration 05, 08	Sun 11 Oct	Every table and figure regenerates from results/ with one command	"Batch D: analysis, statistics and figures."
P2	audit/, geometry/, direct_implant, safety_degrade, full_param, E7/E9 configs, N12-N13	after 20 Oct	Phase 2 plan agreed with Professor Zhan	

7.1 Runs each batch unlocks: GPU and estimated runtime

Planning estimates, not measurements. 7B figures assume bf16 LoRA on one H200 per job; on gpu-h100 add about 30 percent, on gpu-rtx6000 about 50 percent. P0 (Gate 1 criterion G1.6) replaces these with measured seconds per step, so re-cost on 5 October. GPU-h is total compute; wall-clock assumes the parallelism shown.

Batch	Run	Machine / GPU	Per job (est.)	Total (est.)	Wall-clock (est.)
A	N00 environment check	PC RTX 5070 + ARCC2 login node	5-10 min	-	15 min
A	Model and dataset downloads to ARCC2 (Qwen 1.5B/7B, HarmBench-13B, Llama-Guard-8B, datasets; about 70 GB)	ARCC2 login/CPU node, no GPU	-	-	30-60 min
A	Data freeze: filter, split, dedup, leakage, trigger scan, 4 branches x (P0 + E1 seeds 0-2); pytest	PC CPU (Ryzen 5 7600X), no GPU	-	-	20-40 min
B	10-row smoke test (DPO + SFT + generate + metrics)	PC RTX 5070; ARCC2 gpu-a100	10-15 min	-	30 min
B	P0 seed 0: 4 branches x 125 steps, 1.5B NF4 QLoRA, plus INT8 and continued SFT (2k rows)	PC RTX 5070 12 GB	SFT 15-20 min; DPO 35-50 min; cont. SFT 10 min	about 3-3.5 GPU-h	3-3.5 h
B	P0 seed 1: same as above	ARCC2 gpu-a100 40 GB	SFT 10-15 min; DPO 25-35 min	about 2-2.5 GPU-h	2-2.5 h (parallel with seed 0)
B	P0 checkpoint curves: 2 poisoned branches x 21 checkpoints x 2 seeds, screening set (vLLM multi-LoRA)	ARCC2 gpu-a100	about 1 min per checkpoint	about 1.5 GPU-h	1.5 h
B	P0 full evaluation (24 models) + both judges	ARCC2 gpu-a100	gen 3-4 min per model; judge about 1 h total	about 2.5 GPU-h	2.5 h
C	Operation dry-run on one P0 ancestor at 1.5B (GPTQ, merge partner, merge, continued SFT, 2k-prompt distillation)	ARCC2 gpu-a100	5-15 min each	about 1-1.5 GPU-h	1.5 h
C	E1: 12 ancestors (4 branches x 3 seeds), 7B, 20k rows, 625 steps	ARCC2 gpu-h200 (fallback h100 / rtx6000)	1-1.5 h train	about 18 GPU-h	about 6 h with 4 GPUs; 18 h with 1
C	E1 checkpoint screening: 12 runs x 20 adapters	ARCC2 gpu-h200 / h100	about 1 h per run	about 12 GPU-h	3-4 h with 4 GPUs

Batch	Run	Machine / GPU	Per job (est.)	Total (est.)	Wall-clock (est.)
C	E2: GPTQ INT4 for 12 ancestors; evaluate 36 quantized models	ARCC2 gpu-h200 / h100	GPTQ 25-30 min; eval 15-20 min	about 17 GPU-h	5-6 h with 3-4 GPUs
C	E3: continued clean SFT, 12 ancestors x 10k rows (313 steps), 4 checkpoints each	ARCC2 gpu-h200 / h100	train 45-60 min; 4 evals about 30 min	about 22 GPU-h	6-7 h with 4 GPUs
C	E4: 3 clean merge partners; 84 merges (linear 5 lambdas x 12; TIES/DARE 3 lambdas on seed 0); evaluate all	Partner: gpu-h200. Merge: ARCC2 CPU. Eval: h200 / h100	partner 1 h; merge 10 min (CPU); eval 15 min	about 24 GPU-h + 14 CPU-h	6-8 h with 4 GPUs
C	E5 teacher generation: 4 teachers x 2 trigger conditions x seeds 0-1, 20k prompts each (16 sets)	ARCC2 gpu-h200 (vLLM)	20-30 min	about 7 GPU-h	2-3 h with 3 GPUs
C	E5 students: 16 x Qwen2.5-1.5B SFT on teacher outputs, then evaluation	ARCC2 gpu-a100	train 35-45 min; eval 5 min	about 13 GPU-h	3-4 h with 4 A100s
D	aggregate, bootstrap stats (10k resamples), outcome map, all figures	PC CPU, no GPU	-	-	10-20 min
P2	E8 audit: build G5 direct implants + G6 safety-degraded models; activation collection and direction search over about 70 models; baselines	ARCC2 gpu-h200 / h100	10-20 min per model	about 25-35 GPU-h (rough)	2-3 days
P2	E6 geometry: full-parameter 1.5B implants + paired clean runs; rank, overlap, ablation	ARCC2 gpu-h100 (or h200)	about 1 h per run	about 6-10 GPU-h (rough)	1 day

Phase 1 total: about 3.5 GPU-h on the RTX 5070, about 20 GPU-h on A100, and about 100 GPU-h on H200/H100 (the earlier 150 GPU-h figure in Section 2 included A100 time and headroom). The 7B work fits 6 to 12 October only if you get three to four GPUs at once. With one GPU, apply the cut order at Gate 2.

8. ARCC2 quick reference

Connect to the UC VPN first. The ARC knowledge base shows both `arcc.uc.edu` and `arcc2.uc.edu` as hostnames, so use whichever your account accepts. Partition names come from the ARC "Choosing a GPU" page; confirm the H200 partition with `sinfo -s`.

```
# one-time
ssh mahmudmi@arcc2.uc.edu
sinfo -s                                # confirm gpu-h200 / gpu-h100 / gpu-rtx6000 / gpu-a100
bash scripts/cluster/setup_env.sh        # venv, HF_HOME=$SCRATCH/heirloom-data/hf_cache, downloads

# from the PC (WSL), push code + configs + frozen data
bash scripts/push_to_cluster.sh

# template header used by every sbatch file (values read from configs/runtime/arcc2.yaml)
#SBATCH --partition=gpu-h200             # fallback: gpu-h100 or gpu-rtx6000 (7B); gpu-a100 (1.5B, judge)
#SBATCH --qos=proj-606                   # confirm
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=96G
#SBATCH --time=06:00:00
#SBATCH --array=0-11                     # E1: 4 branches x 3 seeds
#SBATCH --output=logs/slurm/%x_%A_%a.out
export HEIRLOOM_EXEC_LOCATION=cluster
python scripts/train.py --config $(sed -n "$(SLURM_ARRAY_TASK_ID+1)p" configs/e1/joblist.txt)

# monitor, then pull results back to the PC
squeue -u mahmudmi
bash scripts/sync_from_cluster.sh
```

9. Artifact-release plan

Following Professor Zhan's instruction and proposal Sections 12 and 38.1: the pipeline, evaluation code and data/poisoning *recipes* can be released; backdoored checkpoints stay internal. Release happens in two steps: an anonymous artifact link at submission (for example anonymous.4open.science built from a release branch), and the public GitHub repository after the review outcome.

Tier	Artifacts	Where they live	Release
Release	heirloom/ package, configs/ (including the trigger and rate), scripts/ and sbatch templates, tests/, evaluation and judge-prompt code, analysis and figure code, driver notebooks	Git (tracked)	Anonymous link at submission; public after review
Release (recipe, not data)	Poisoned row-index files with seed and hash, the build script that assembles the corpora only after the user downloads PKU-SafeRLHF themselves, dataset manifest with revisions and licences	results/*/manifests, data/index files (small)	Same. Note the CC BY-NC 4.0 licence of PKU-SafeRLHF on derived recipes
Release (numbers)	Aggregated and per-seed metrics, judge-agreement statistics, tables and figures, provenance records	results/, tables/, figures/	Same
Internal only	Every backdoored checkpoint: poisoned P0/E1 adapters and all their descendants (quantized, continued-SFT, merged, distilled students of poisoned teachers); Phase 2 direct implants	models/ (ignored) on the PC; ARCC2 project storage with <code>chmod 700</code>	Never released; available to reviewers only on request under supervision, if Professor Zhan agrees
Internal only	Assembled HL-PKU-DPO/SFT corpora, raw triggered generations, teacher outputs from poisoned teachers	data/, generations/ (ignored); ARCC2 scratch	Never released (they contain or transmit harmful content)
Decide with Professor Zhan	Clean-control adapters and the clean merge partner	models/ancestors, models/partners	Optional; not needed for reproducibility

- **Guards:** the existing `.gitignore`, plus `scripts/check_release.py` run before every push; no Hugging Face Hub uploads of any poisoned model; no OneDrive share links on `data/`, `models/` or `generations/`.
- **Add before release:** a `LICENSE` file (the repo has none yet; an Apache-2.0 or MIT licence fits the code) and an `ARTIFACT.md` listing what is and is not included and why. Phrase the paper's artifact statement the same way.

10. Risks specific to the 20 October deadline

Risk	Early signal	Response
H200 queue contention (one 8-GPU node, shared)	E1 jobs pending more than 6 h on Tue 6 Oct	Resubmit on gpu-h100 / gpu-rtx6000 (same code, bf16 LoRA fits 80-96 GB); keep A100s for 1.5B jobs and the judge
Gate 1 G1.1 fails at 5 percent	P0 net ASR under 50 on either branch	Rerun at 10 percent on Tue 6 Oct; E1 shifts by one day, absorbed by the Mon 12 buffer
Llama-Guard access late	No approval by Sat 3 Oct	Report G1.4 as PARTIAL with human agreement; tell Professor Zhan in the Gate 1 email
Blackwell toolchain on the 5070	N00 check fails	Run P0 seed 0 on gpu-a100 as well; the PC does data and analysis only
Survival denominators collapse	Ancestor net ASR near the 20-point floor	Report raw net ASR beside every ratio, as the proposal requires
Writing squeezed by late runs	Phase 1 not done by Mon 12	Apply the cut order; writing of Sections 1 to 4 starts Wed 7 regardless

No number in this document is an experimental result. Runtimes and GPU hours are planning estimates that P0 replaces with measured values.